

Verité Ingestion, Matching & Scoring

Detailed Engineering Specification for LLM Ingestion

This document contains the exact technical specification of the Verité record matching pipeline. It outlines the filtering (blocking) rules, the smart-name heuristics hierarchy, individual factor distance calculations, year slop tolerance algorithms, name frequency rarity formulas, and impact multipliers.

1. Phase 1: Ingestion & Demographic Blocking (MakeBlockedMentions)

The blocking filter (app.js) executes an initial demographic check on all raw database mentions to eliminate impossible candidates before running downstream Jaro-Winkler or phonetic matches:

- * **Source Filtering:** Candidates whose raw source value is not present in the active sources array are skipped.
- * **Gender compatibility:** If both target and mention have gender data, their first letters (lowercased) must match: `target_gender[0].toLowerCase() == mention_gender[0].toLowerCase()`
- * **Race compatibility:** If both target and mention have race data, the target race must match either candidate's `norm_race` or `raw_race`: `target_race == norm_race || target_race == race`
- * **Time Span Boundary Filter:** If both `birth_year` and `source_year` are present, the absolute difference cannot exceed 75 years: `|target_birth_year - mention_source_year| <= 75`. Handled by splitting strings on ':' and '-' and parsing as floats.

2. Phase 2: Scoring Engine (ScoreMentions & ScoreFactor)

2.1 Smart Name Matching Heuristics Cascade

In Smart Name Mode, separate string similarity metrics are disabled in favor of a cascaded name evaluation order (precedence rules):

Strategy (Heuristic)	Base Score	Trigger Criteria & Logical Condition
Exact Match	1.00	<code>StrEq(first, first) && StrEq(middle, middle) && StrEq(last, last)</code>
First & Last Match	0.95	<code>StrEq(first, first) && StrEq(last, last)</code>
Initials Match	0.90	<code>first[0] == first[0] && last[0] == last[0]</code> (case-insensitive)
Nickname Match	0.90	<code>StrEq(GetNickname(first_target), first_mention) && StrEq(last, last)</code>
Jaro-Winkler Double	0.70	<code>JW(norm_first, norm_first) > 0.85 && JW(last, last) > 0.85</code>
NYSIIS phonetic	0.60	<code>StrEq(NYSIIS(last_target), NYSIIS(last_mention))</code>
Soundex phonetic	0.50	<code>StrEq(Soundex(last_target), Soundex(last_mention))</code>

Smart Name Score Multipliers: Adjusts based on the target `first_name` impact setting. If `impact > 0`, the final smart score is multiplied by 2 (`score * 2`). If `impact < 0`, it is halved (`score * 0.5`). In this mode, a rare name test is automatically forced on the `last_name` factor.

2.2 Independent Factor Matching (Non-Name & Slop Years)

If Smart Name mode is disabled, name fields are matched independently. Slop years (`birth_year`, `death_year`) are evaluated using slop band thresholds: `|target - mention| <= slop`. If matched, the score is multiplied by the slop coefficient: Slop 0 (1.0), Slop 1 (0.8), Slop 2 (0.7), Slop 3 (0.6), Slop 5 (0.5), Slop 10 (0.3), Slop 20 (0.2). Other factors use Jaro-Winkler distance if fuzzy match is set.

2.3 Mathematical Formula: Name Rarity Score

When the 'rare' parameter is enabled (or automatically triggered for last_name in Smart Name Mode), the rarity score is calculated relative to corpus frequencies using the following formula:

$$u = (\text{count} + 0.5) / (\text{total} + 1)$$
$$\text{rarity_score} = \text{Max}(0.0, \text{Min}(0.3, 0.0225 * \text{Log2}(0.95 / u) - 0.1))$$

Where 'count' is the frequency of the name in the dataset corpus, and 'total' is the sum of all name frequencies in that category (first name corpus vs. last name corpus). The result is strictly clamped to [0.0, 0.3].

2.4 Factor Impact Modifiers

All active, non-ignored factors (both name and non-name) scale their individual scores based on the factor weight modifier: Positive Impact (double score), Negative Impact (halve score), Neutral Impact (no change).